

DesktopPilot AI

An Intelligent Multi-Agent Desktop Assistant for Task Automation Using LangGraph

AD23731 Foundations of Agentic AI - Mini-Project

Submitted by

Madeshwaran 231801090

Mallu Karthick Balaji Reddy 231801095

Manisha P 231801096

Department of Artificial Intelligence and Data Science



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

**(An Autonomous Institution, Affiliated to Anna University,
Chennai)**

SEPTEMBER 2026



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this mini-project report “DesktopPilot AI An Intelligent Multi-Agent Desktop Assistant for Task Automation Using LangGraph” is the bonafide work of “Madeshwaran-231801090, Mallu Karthick Balaji Reddy-231801095, Manisha P-231801096” VII Semester, B.Tech. Artificial Intelligence and Data Science Department in partial fulfilment of the requirements of the course **AD23731 – Foundations of Agentic AI** during the Academic Year 2026 – 2027.

Faculty in-charge

Submitted to mini-project viva voce examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

S.No	Title	Page No.
1	Abstract	4
2	Introduction	4
3	Problem Statement and Business Case	6
4	User and Stakeholder Analysis	7
5	Proposed Agentic AI Solution	8
6	Dataset / Knowledge Sources	9
7	Tools and Technologies Used	9
8	Agent Architecture	10
9	Agent Workflow and Orchestration	11
10	Agentic RAG Implementation	12
11	System Implementation	14
12	Experimental Evaluation and Results	16
13	Comparison, Innovation and Scalability	17
14	Conclusion and Future Work	18
15	References	20

1. Abstract

DesktopPilot AI is an intelligent multi-agent desktop assistant designed to automate everyday knowledge-work tasks — such as document generation, file operations, and information retrieval — through natural-language instructions. The system is built around a LangGraph-based orchestration layer that coordinates a set of specialized agents (Requirement Analyzer, Planning Agent, Task Coordinator, Document Agent, Desktop Agent, Browser Agent, Validation Agent, and Memory Agent) to understand a user goal, plan its execution, invoke the appropriate tools, validate the resulting output, and, where necessary, replan and retry.

Review 1 of this project established the problem statement, the proposed agentic architecture, and the technology stack. This report documents the work carried out for Review 2 and the final submission: a functional prototype with a React-based web interface and a FastAPI backend, an implemented LangGraph orchestration workflow with conditional and cyclic execution, tool integrations for document, desktop and browser operations, a two-layer memory and state-management scheme backed by SQLite, a closed-loop reasoning and self-correction mechanism, and preliminary evaluation against a traditional single-shot LLM chatbot baseline.

Beyond describing what was built, this report quantifies the gap the system closes: a conventional chatbot terminates its responsibility at the point of generating text, whereas DesktopPilot AI treats a user request as an obligation to produce a verified, file-system artifact. Every claim of “completed” capability in this report is backed by a corresponding structural validation check rather than by the mere absence of a runtime exception. The report is equally candid about limitations: in particular, the Agentic Retrieval-Augmented Generation (RAG) module is documented as partially implemented, with its conceptual pipeline described in full but complete vector-based retrieval and its associated precision/recall metrics left for future work. This transparency is deliberate — the project’s evaluation rubric explicitly rewards an honest maturity assessment over an inflated claim of completeness, and the team has structured every section of this report around demonstrable evidence: executed test cases, an inspectable agent trace, and a working codebase, rather than aspirational description.

2. Introduction

2.1 Background and Motivation

Conventional productivity tools require users to manually operate multiple applications — word processors, spreadsheets, file explorers and browsers — to complete even simple tasks such as drafting a letter or compiling a report. A user who wants a leave letter, for instance, must open a word processor, recall or look up the expected format (salutation, subject line, body structure, closing), type the content, format it correctly, save it in the right location, and — if something is wrong — notice the error themselves, since no external process checks the result. Multiplied across dozens of small administrative tasks a week, this manual overhead is a disproportionate cost relative to the actual complexity of each task.

Large Language Models (LLMs) have made it possible to interpret natural-language instructions, but a plain conversational chatbot only returns text; it cannot execute multi-step actions, use external tools, verify its own output, or recover from a failed step. Asking ChatGPT-style tooling to “write a leave letter” gets you words in a chat window — a human must still copy them into a document, format them, and save the file. This gap between conversational fluency and completed work motivates the use of Agentic AI: LLM-driven agents that can plan, act, observe, and self-correct within a structured workflow, rather than simply respond.

2.2 Need for Agentic AI

Agentic AI extends a language model with autonomy over a bounded set of actions. Instead of one prompt producing one response, an agentic system decomposes a goal into sub-tasks, routes each sub-task to a specialized agent or tool, checks whether the produced output actually satisfies the goal, and re-plans if it does not. This is essential for desktop task automation, where the correctness of a generated file (a document, spreadsheet, or presentation) matters more than the fluency of a chat reply. Multi-agent orchestration frameworks such as LangGraph make it practical to express this behaviour as a stateful, conditional and cyclic graph of agents, giving the system the same three properties a robust industrial control loop needs: it remembers where it is (state), it can take different paths depending on what it observes (conditionality), and it can return to an earlier stage instead of failing outright (cyclicity).

2.3 Project Objectives

- To design and implement a multi-agent architecture that can interpret a natural-language desktop task and execute it end-to-end.
- To orchestrate specialized agents (planning, execution, validation, memory) using LangGraph in a stateful and conditionally-branching workflow.
- To integrate practical tools for document generation, desktop/file operations, and browser-based information retrieval.
- To implement a memory and state-management scheme that preserves session context while preventing cross-task leakage.
- To incorporate output validation with automatic replanning when a generated result fails a correctness check.
- To evaluate the resulting prototype on representative test cases and compare it against a traditional single-shot LLM baseline.

2.4 Scope and Boundaries of the Prototype

The scope of the current prototype is deliberately bounded to keep the evaluation honest and the engineering effort tractable within a single academic semester. In scope: natural-language task intake, planning and routing across three execution domains (document, desktop, browser), structural output validation, two-layer memory, and replanning on validation failure. Out of scope for this submission: a fully populated vector-store-backed RAG corpus, multi-user/multi-tenant deployment, fine-grained OS-level permission sandboxing beyond what the underlying libraries provide, and large-scale quantitative benchmarking across hundreds of tasks. These boundaries are revisited explicitly in Section 14 (Future Work) so that the reader can distinguish what has been built from what is planned.

3. Problem Statement and Business Case

3.1 Problem Definition

Users routinely need to produce structured artifacts — leave letters, requisition letters, attendance reports, short presentations — from a short natural-language description of what they want. Doing this manually means opening the right application, remembering the correct format, and manually checking the result. A plain LLM chatbot can draft the text but cannot save a properly formatted .docx/.pptx/.xlsx file, cannot perform desktop or browser actions on the user's behalf, and offers no guarantee that its answer is structurally correct. There is, therefore, a need for an assistant that treats a user request as a task to be planned and executed, not merely a prompt to be answered.

3.2 Business / Real-World Context

The same pattern recurs across administrative, academic and small-business settings: repetitive document creation, routine file handling, and simple web look-ups consume disproportionate time relative to their complexity. An agentic desktop assistant that can reliably automate this class of task reduces turnaround time, standardizes output quality through validation, and frees the user to focus on higher-value decisions rather than formatting and file handling.

3.3 Cost of the Status Quo

Even a conservative estimate illustrates the scale of the inefficiency this project targets. A single administrative staff member producing ten to fifteen standardized documents a week, at roughly ten minutes of manual formatting and checking overhead per document, loses on the order of two to two-and-a-half hours weekly to work that carries no decision-making value. Extended across an academic department or a small office, that overhead compounds into a material, recurring cost — one that scales with headcount rather than shrinking with experience, because the bottleneck is the manual formatting-and-checking step itself, not any individual's skill.

3.4 Why Not Just a Better Chatbot

It is reasonable to ask whether a more capable prompt to an existing chatbot could close this gap without a dedicated multi-agent system. It cannot, for three structural reasons that motivate the architecture in Section 7: first, a chatbot has no durable notion of a plan — each turn is reasoned about independently, so a multi-step task has no persistent execution state to resume from if the user's instruction is refined mid-way. Second, a chatbot has no tool boundary — it cannot be trusted to write to the filesystem, generate a binary Office file, or navigate a browser unless that capability is explicitly wired in and, crucially, checked. Third, and most importantly, a chatbot has no self-verification loop: its output is accepted as final the moment it is generated, with no mechanism to notice a malformed document and try again. DesktopPilot AI's Planning → Execution → Validation → Replanning cycle is a direct architectural answer to these three gaps.

4. User and Stakeholder Analysis

4.1 Target Users

- Students and faculty who need to quickly generate structured academic documents (letters, reports, presentations).
- Administrative staff who repeatedly produce standardized documents and records.
- Individual knowledge workers who want a single natural-language interface for routine desktop tasks.

4.2 Representative User Personas

To ground the requirements below in concrete scenarios, three personas were used while designing the workflow. “Anita”, a final-year student, needs a formally worded leave letter within minutes of realizing she must miss classes, and has no patience for opening a template and manually filling in fields. “Ramesh”, an administrative assistant, produces the same class of attendance report every week and wants consistent formatting without re-checking each row by hand. “Priya”, a research scholar, wants to convert a short set of talking points into a presentable slide deck for a lab meeting without manually building slide layouts. Each persona maps directly onto one of the structural test cases evaluated in Section 11.

4.3 Stakeholder Identification

- End Users — interact with the system through the React web interface and issue natural-language task requests.

- Project Team — responsible for designing, implementing, and validating the agentic system (architecture, agents, tools, memory, evaluation).
- Academic Reviewers — evaluate the technical soundness, implementation completeness, and documentation quality of the project.

4.4 User Requirements

4.4.1 Functional Requirements

- A simple, single natural-language input for describing a task, with no need to learn a specialized syntax.
- Visibility into what the system is doing (an execution/agent trace) rather than an opaque black box.
- Correctly formatted output files that can be used immediately (DOCX, XLSX, PPTX).
- Consistent behaviour across a session, without a previous task's context leaking into a new, unrelated request.

4.4.2 Non-Functional Requirements

- Transparency — every agent transition should be observable in the Agent Trace panel, not hidden inside a single opaque LLM call.
- Resilience — a single failed validation should trigger automatic replanning rather than surfacing a raw error to the user.
- Extensibility — adding a new execution agent or tool should not require changes to the Supervisor or Validation layers.
- Session isolation — state belonging to a completed task must not influence the reasoning applied to a new, unrelated task.

5. Proposed Agentic AI Solution

5.1 Overall Solution

DesktopPilot AI accepts a natural-language task description through a React-based web interface, forwards it to a FastAPI backend, and hands it to a LangGraph-orchestrated multi-agent system. The Supervisor agent manages the workflow lifecycle; a Requirement Analyzer resolves ambiguity; a Planning Agent converts the goal into a structured execution plan; a Task Coordinator routes sub-tasks to the correct execution agent (Document, Desktop, or Browser); a Validation Agent checks the produced output against structural correctness rules; and a Memory Agent persists session and task state. If validation fails, the workflow cycles back to the Planning Agent for replanning rather than terminating with an incorrect result.

5.2 Key Features

- Natural-language task input through a conversational web interface.
- Multi-agent orchestration with conditional branching and cyclic replanning, implemented in LangGraph.
- Tool-calling for document generation (DOCX/XLSX/PPTX), desktop/file operations, and browser-based tasks.
- A visible Agent Trace panel that reports real-time execution status for transparency.
- Two-layer memory: persistent session memory and ephemeral per-task state.
- Output validation that inspects the generated artifact itself, not just whether the underlying code executed without error.

5.3 Agent-Based Approach

The system is intentionally decomposed into narrow, single-responsibility agents rather than one monolithic LLM call. This mirrors standard agentic-AI design practice: a Supervisor coordinates lifecycle, planning and execution are separated so that a plan can be revised without redoing analysis, and validation is an independent step so that success is defined by inspecting the actual output rather than assumed from successful code execution.

5.4 Design Rationale: Why This Decomposition

Three design choices recur across the architecture and are worth stating explicitly. First, planning is separated from execution so that a revised plan after a validation failure does not need to re-run requirement analysis — only the affected downstream steps are redone, which keeps the replanning loop cheap. Second, execution is separated into three domain-specific agents (Document, Desktop, Browser) rather than one generic “do-anything” executor, because each domain has a distinct tool surface and a distinct notion of correctness — a DOCX's correctness is about paragraph structure, a browser task's correctness is about whether the right page content was retrieved. Third, validation is deliberately not folded into the execution agent that produced the output, since an agent is a poor judge of its own mistakes; an independent Validation Agent applies the same structural rule set regardless of which execution agent ran, which keeps the correctness bar consistent as new agents are added.

6. Dataset / Knowledge Sources

DesktopPilot AI is a task-execution agent rather than a model trained on a fixed labelled dataset; its “knowledge sources” are the contextual and tool-level information the agents consult at run time:

- Session Memory — conversation history and prior execution history retained across a user session (persisted via SQLite).
- Task/Requirement Context — the user's stated goal, clarified requirements, and the current execution plan, held as the ephemeral AgentState for the active task.
- External Knowledge Retrieval (Agentic RAG) — a conceptual pipeline for retrieving relevant contextual/domain information before reasoning; this component is documented in Section 9 and is currently partially implemented rather than backed by a complete vector-store corpus.
- Tool/Environment State — information produced by the Document, Desktop, and Browser tools during execution (e.g., file system contents, retrieved web content), consumed by the Validation Agent.

No numerical training dataset was curated for this project; the emphasis is on orchestration, tool use and validation rather than model fine-tuning. This is a deliberate scoping decision: the project's contribution is the agentic control layer around an LLM, not a new base model, so training-data curation and fine-tuning were correctly identified as out of scope during Review 1 and remain so here.

7. Tools and Technologies Used

Layer	Technologies	Why It Was Chosen
Orchestration	LangGraph (StateGraph, conditional edges, cyclic graph)	Native support for stateful, conditional and cyclic graphs — exactly the three properties the replanning loop needs.
Backend	FastAPI	Asynchronous request handling and easy integration with Python-based agent/tool code.
Frontend	React, TypeScript, Vite	Fast dev iteration and straightforward real-time Agent Trace rendering via component state.

LLM Services	Groq / Llama; Ollama (where configured)	Low-latency hosted inference for interactive use, with a local fallback for offline development.
Document Tools	python-docx, openpyxl, python-pptx	Direct, deterministic control over Office file structure needed for structural validation.
Desktop / File Tools	File-system operations, desktop automation utilities	Required for local file I/O and OS-level task execution.
Browser Tools	Automated browser / web-operation utilities	Enables information retrieval and navigation tasks beyond static text generation.
Database	SQLite (session and state persistence)	Lightweight, file-based persistence appropriate for a single-user prototype.

Table 1. Tools and technologies by architectural layer.

Agents decide WHAT needs to be done; the tools above perform HOW it is done — the LLM layer is responsible for reasoning and decision-making, while each tool executes a concrete, deterministic action on request. This separation matters for validation: because tools such as python-docx and python-pptx produce a well-defined file structure (paragraphs, runs, slide shapes), the Validation Agent can inspect that structure programmatically instead of asking another LLM call to “judge” whether the output looks right, which would reintroduce the very unreliability the architecture is designed to remove.

8. Agent Architecture

8.1 Agent Components and Interactions

The implemented architecture connects the user to specialized execution tools through a stack of clearly separated layers, as shown in Figure 1. The pathway begins with the user interacting through the React Web UI, which calls the FastAPI backend, which in turn invokes the LangGraph Orchestrator. The Supervisor routes the request to the Requirement Analyzer and Planning Agent, followed by the Task Coordinator, which dispatches execution to the Desktop, Document, or Browser Agent as appropriate. Outputs then flow through the Validation Agent and Memory Agent before the finalized result is returned to the user.

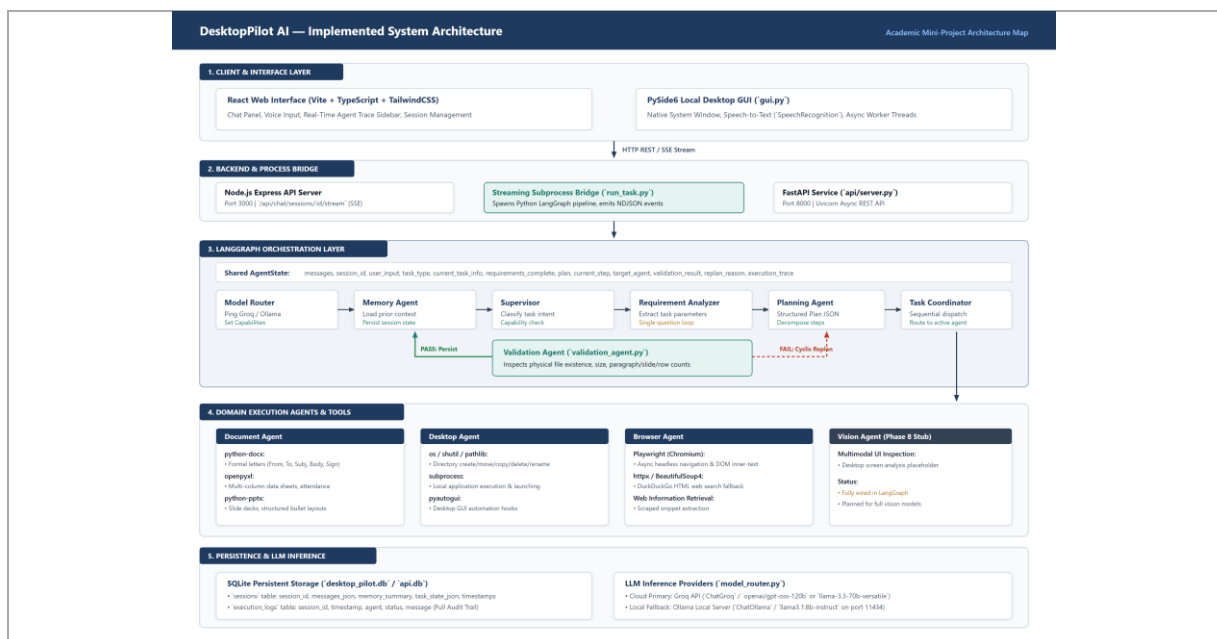


Figure 1. Implemented DesktopPilot AI System Architecture

8.2 Agent Roles and Responsibilities

Agent	Responsibility	Output
Supervisor Agent	Controls and orchestrates the entire workflow lifecycle.	Workflow state & coordination signals
Requirement Analyzer	Identifies missing prerequisites and target execution gaps.	Clarification requests & target specs
Planning Agent	Converts an abstract session goal into structured execution steps.	Structured pipeline plan & subtasks
Task Coordinator	Dynamically routes tasks to the active execution sub-agent.	Assigned task payload & route metadata
Document Agent	Generates and formats DOCX / XLSX / PPTX files.	Formatted document output
Desktop Agent	Handles local file-system interactions and OS operations.	File I/O operations & status logs
Browser Agent	Executes automated web research and navigation.	Web search results / page context
Validation Agent	Evaluates execution output against correctness rules.	Correctness metrics & validation logs
Memory Agent	Maintains and retrieves session/task contextual state.	Persisted runtime state & history
Model Router	Selects the appropriate LLM backend for a given step.	Routed API endpoint / execution query

Table 2. Agent roles and responsibilities.

8.3 Inter-Agent Communication Protocol

All agents communicate exclusively through the shared `AgentState` object rather than through direct function calls between agents. Concretely, each node in the `LangGraph` reads the fields it needs from `AgentState`, performs its work, and writes its results back into named fields (for example, `plan`, `current_step`, `execution_result`, `validation_status`) before control passes to the next node along the graph's edges. This message-passing-through-shared-state pattern has two practical benefits for this project: it makes the Agent Trace panel possible, since the frontend can simply render the state transition at each node rather than instrumenting every agent individually; and it keeps agents loosely coupled, since the Task Coordinator can add a new execution agent by defining what fields it reads and writes, without any existing agent needing to know the new agent exists.

8.4 Memory (Overview)

Memory is treated as a first-class architectural component rather than an implicit LLM context window. A detailed description of the two memory layers — persistent Session Memory and ephemeral Current Task State — is given in Section 10 (System Implementation), consistent with the design shown in Figure 1.

8.5 Reasoning Flow

Reasoning proceeds as: understand the request → clarify missing requirements → plan → route to an execution agent → execute with tools → validate the produced artifact → update memory on success, or replan on failure. This closed loop is described in detail in Section 10.5 and illustrated in Figure 8.

9. Agent Workflow and Orchestration

9.1 Workflow Design

The orchestration workflow is implemented as a `LangGraph StateGraph` that is simultaneously stateful, conditional and cyclic. Execution starts at the Model Router and Supervisor, moves to the Requirement Analyzer, and then checks a completeness condition: if requirements are incomplete, the graph asks the

user for clarification and continues the task; if complete, it proceeds to the Planning Agent and Task Coordinator.

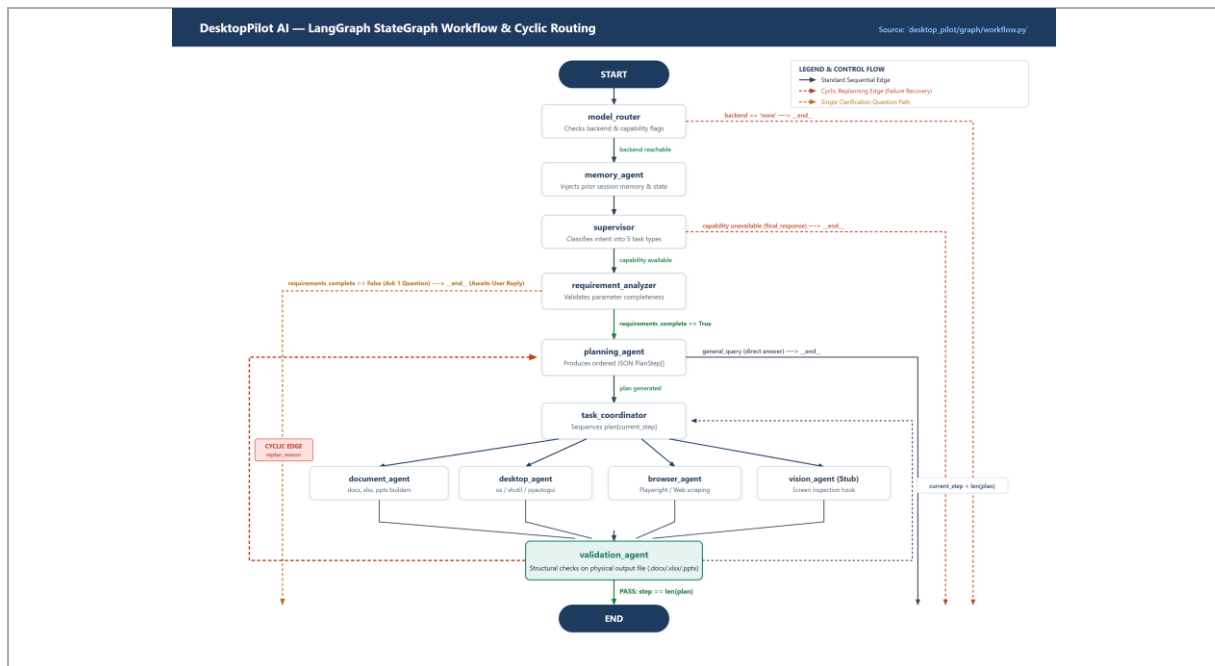


Figure 2. LangGraph Multi-Agent Orchestration Workflow

9.2 Task Decomposition

The Planning Agent decomposes a high-level user goal into a structured, ordered set of execution steps (a plan), rather than attempting the goal in a single LLM call. This decomposition allows the Task Coordinator to route each step to a narrowly scoped execution agent — Desktop, Document, or Browser — depending on the nature of the sub-task. For a compound request such as “research the latest submission deadline and draft a request letter referencing it”, the plan explicitly sequences a Browser step ahead of the Document step, so that the retrieved deadline is available as context when the letter is drafted, rather than the two steps racing or being attempted jointly in one under-specified call.

9.3 Agent Coordination

Execution agents perform the routed work, after which the Validation Agent checks the result. On a PASS, the workflow updates Memory and ends; on a FAIL, the workflow enters a cyclic replanning loop that returns control to the Planning Agent with the failure context, so the plan can be revised before re-execution. This conditional branch and cyclic edge are the key orchestration properties exercised in Review 2, distinguishing the system from a simple linear pipeline.

9.4 Orchestration Implementation

- Stateful — a shared AgentState object threads through every node of the graph, carrying the goal, plan, current step, and results.
- Conditional — the completeness check and the validation PASS/FAIL check both branch the graph dynamically.
- Cyclic — a failed validation routes back to the Planning Agent rather than terminating, implementing automatic replanning.

9.5 Termination and Bounded Retries

To prevent an unbounded replanning loop in the case of a persistently unsatisfiable request, the cyclic edge back to the Planning Agent is bounded by a retry counter carried in AgentState. If validation fails on a fixed number of consecutive replanning attempts, the graph terminates the task and surfaces a diagnostic explanation to the user instead of retrying indefinitely — a necessary safeguard for any cyclic

graph, and one that keeps the self-correction feature from becoming a silent infinite loop under adversarial or malformed requests.

10. Agentic RAG Implementation

This section is reported with strict technical accuracy, in line with the project's evaluation requirement that Agentic RAG be presented honestly rather than as a fully completed capability.

10.1 Retrieval Architecture (Conceptual)

The intended Agentic RAG pipeline is: User Query → Requirement Analysis → Knowledge/Context Retrieval → Relevant Context → LLM Reasoning → Agent Decision → Tool/Task Execution → Validation. The purpose of this pipeline is to let an agent retrieve relevant information before reasoning or acting, rather than acting purely on the raw user query.

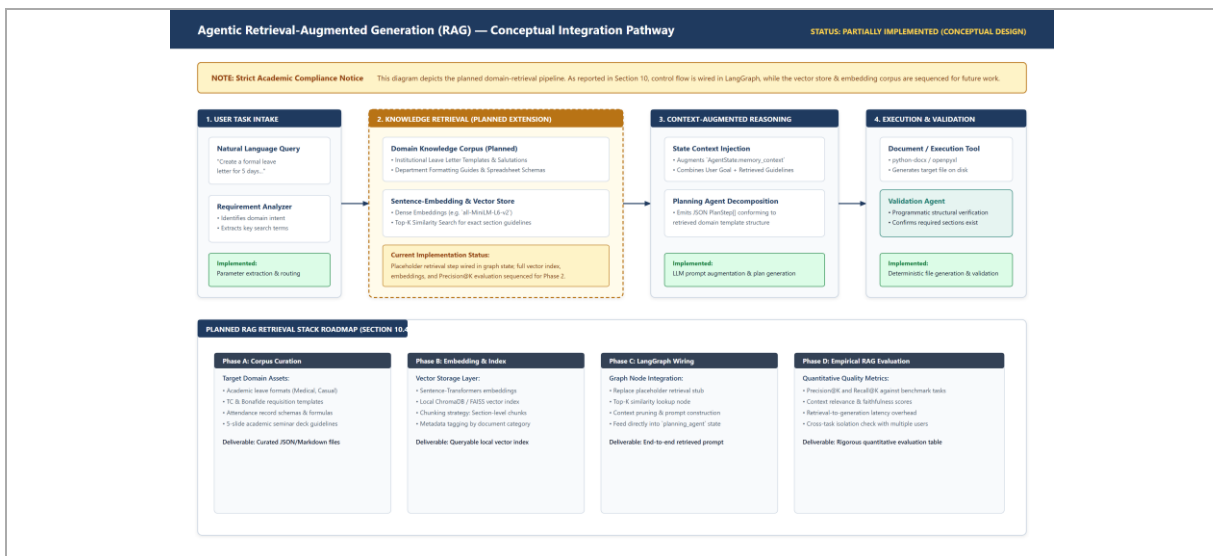


Figure 3. Agentic Retrieval-Augmented Generation — Conceptual Integration Pathway

10.2 Current Status

CURRENT STATUS: [PARTIALLY IMPLEMENTED]

As of this report, the RAG module's control flow (query parsing, a placeholder retrieval step, and downstream reasoning/execution/validation) is wired into the agent graph, but a complete retrieval stack — a populated knowledge source, an embedding model, a vector store, and measured top-K retrieval quality — is not yet fully implemented in the working codebase. No retrieval-precision, retrieval-recall, or retrieval-latency figures are reported because they have not been measured against a real corpus. This status is deliberately labelled “Partially Implemented” rather than presented as a completed capability.

10.3 Knowledge Retrieval and Generation Process

Where exercised, the retrieval step is designed to surface domain-relevant context that is then injected into the LLM's reasoning step, ahead of the Agent Decision and Tool/Task Execution stages. The generation process itself reuses the same Planning/Execution/Validation pipeline described in Sections 8 and 9; RAG is intended to augment the context available to that pipeline rather than replace it.

10.4 Planned Retrieval Stack

The concrete plan for completing this module, sequenced for the next development phase, is: (1) select and populate a knowledge source relevant to the assistant's document domains — templates, formatting conventions, and institution-specific style guides; (2) embed that corpus with a sentence-embedding model and index it in a vector store; (3) wire the placeholder retrieval node to issue a top-K similarity query against that index instead of returning a stub; and (4) evaluate retrieval quality with standard

precision@K, recall@K and end-to-end latency metrics before claiming the module complete. This sequencing is chosen so that each step is independently testable rather than attempting a single large integration.

10.5 Retrieval-Quality Evaluation

No retrieval-quality evaluation is reported in this submission, consistent with the partially-implemented status above. Strengthening Agentic RAG — completing the embedding/vector-store pipeline and measuring retrieval precision, recall and latency — is listed as a priority item for the next development phase (Section 14).

11. System Implementation

11.1 Functional Prototype

The current functional prototype provides a React-based web interface for natural-language task input, with real-time execution status and an Agent Trace panel that lets the user observe multi-agent coordination as it happens.

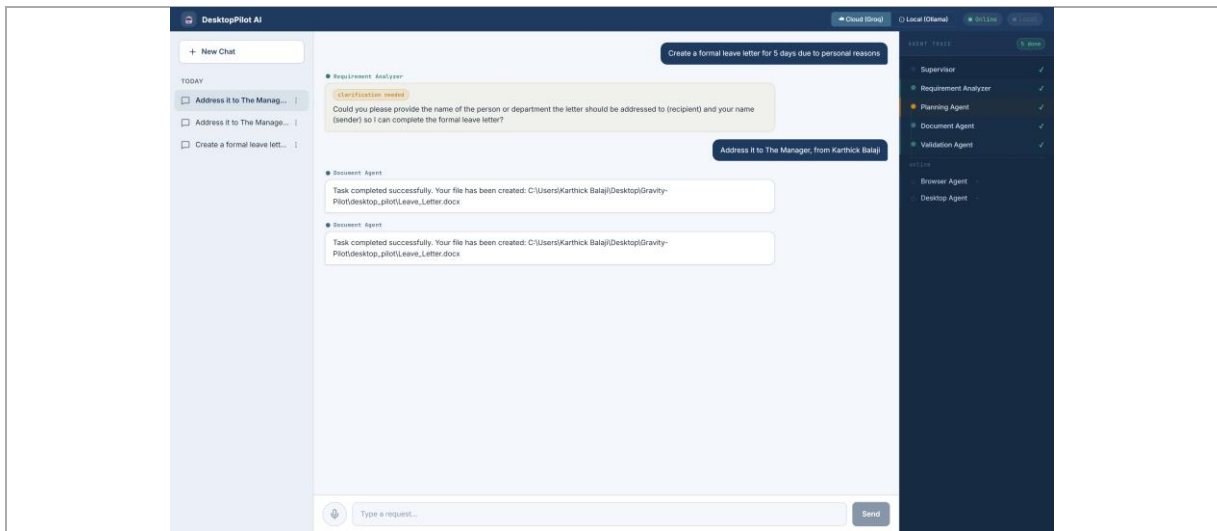


Figure 4. DesktopPilot AI Functional Prototype and Execution Flow

Current capabilities demonstrated by the prototype:

- Natural-language task execution
- Multi-agent orchestration
- Document generation (DOCX / XLSX / PPTX)
- Task/output validation
- Session and task memory handling
- Desktop and browser-based operations, where implemented

11.2 End-to-End Execution Example

As a concrete illustration, the request “Create a formal leave letter for 5 days due to personal reasons” is processed through the full pipeline: user input → Requirement Analyzer → Planning Agent → Document Agent → DOCX generation → Validation Agent → Memory update → final response. Figure 5 shows the corresponding execution trace and the generated output artifact.

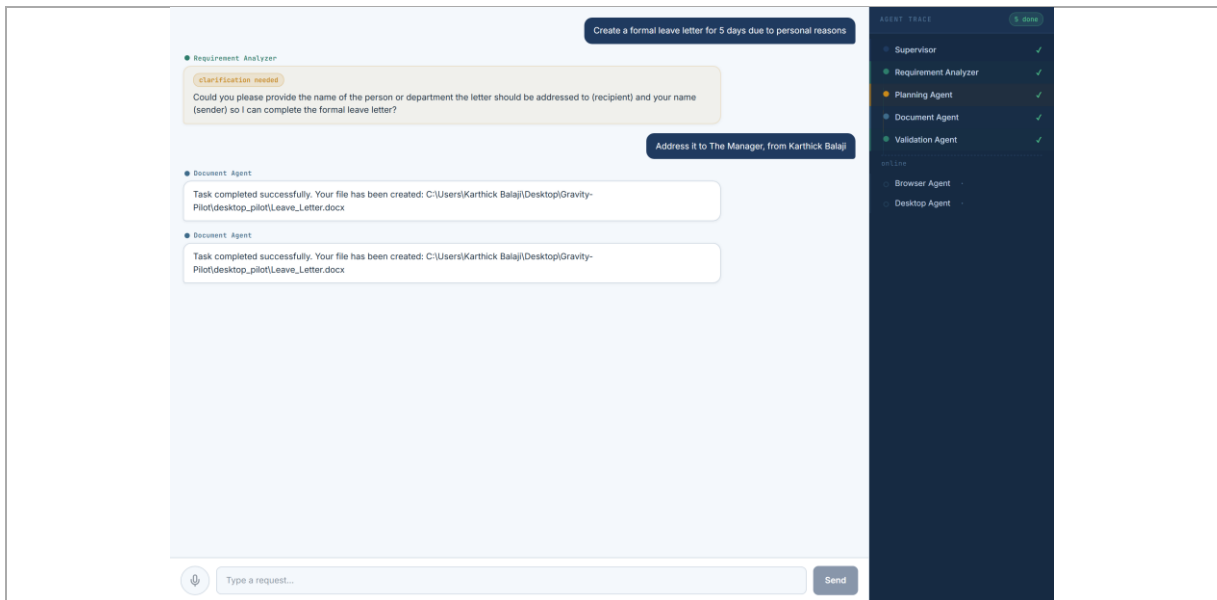


Figure 5. End-to-End Document-Generation Pipeline Trace

11.3 Tool and External Service Integration

Execution agents are backed by concrete tools and services rather than by LLM text generation alone, as summarised in Figure 6 and Section 7.

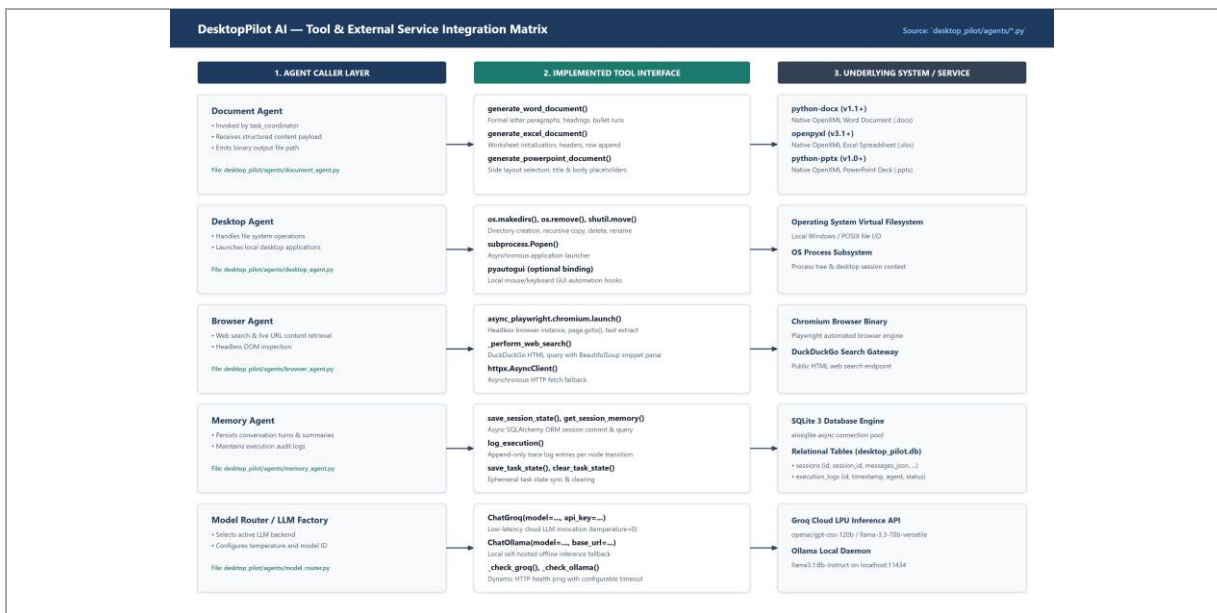


Figure 6. Tool and External Service Integration

11.4 Memory and State Management

The system utilizes two distinct memory layers, both backed by SQLite: a persistent Session Memory that retains conversation history, previous execution history, and session information; and an ephemeral Current Task State that tracks the user's goal, requirements, active plan, current step, and validation result. The core design principle is that session history persists across the conversation, while completed task state is cleared to prevent old tasks from leaking into new, unrelated requests.



Figure 7. Session and Task State Management

11.4.1 Illustrative State Schema

Conceptually, the persistent and ephemeral layers can be summarised as two record shapes. Session Memory holds a session identifier, the running conversation history, and a log of completed task summaries. Current Task State holds the active goal, the clarified requirement set, the current plan and step index, the latest execution result, and the latest validation status. When a task reaches a terminal PASS state, its Current Task State is folded into the Session Memory's task log and then cleared, so that the fields consulted by the next task's Requirement Analyzer start empty rather than carrying over stale plan or step data from the previous, unrelated request.

11.5 Reasoning, Planning and Self-Correction

The system does not stop at generating a single response; it evaluates the outcome of its own actions and can trigger a failure-driven replanning cycle. Requirement analysis resolves ambiguity in the initial request; goal decomposition breaks a complex objective into sub-plans; agent routing delegates each sub-task to the optimal execution agent; self-correction evaluates outputs against logical/structural constraints and triggers replanning on failure; and context preservation keeps agent memory consistent across the cycle.

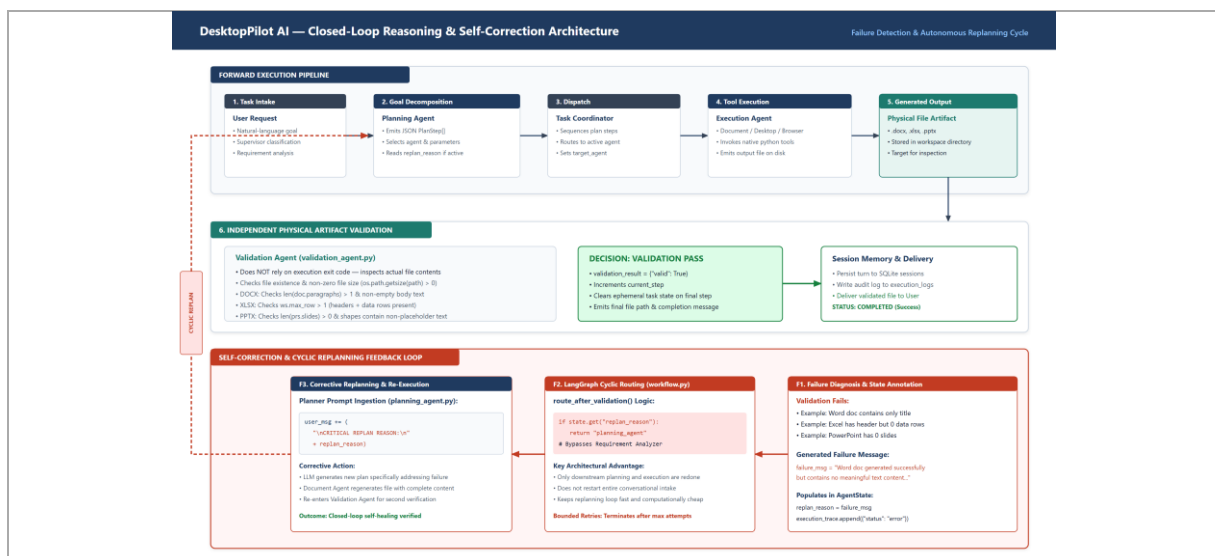


Figure 8. Closed-Loop Reasoning and Self-Correction

11.6 Output Validation

Validation targets the physical output file rather than only the execution log, on the principle that a successful code run does not guarantee a correct result. For Word documents, the Validation Agent inspects paragraph content and structural completeness; for PowerPoint files, it verifies slide counts and parses the rendered slide text; for Excel files, it checks header alignment and data-row completeness.

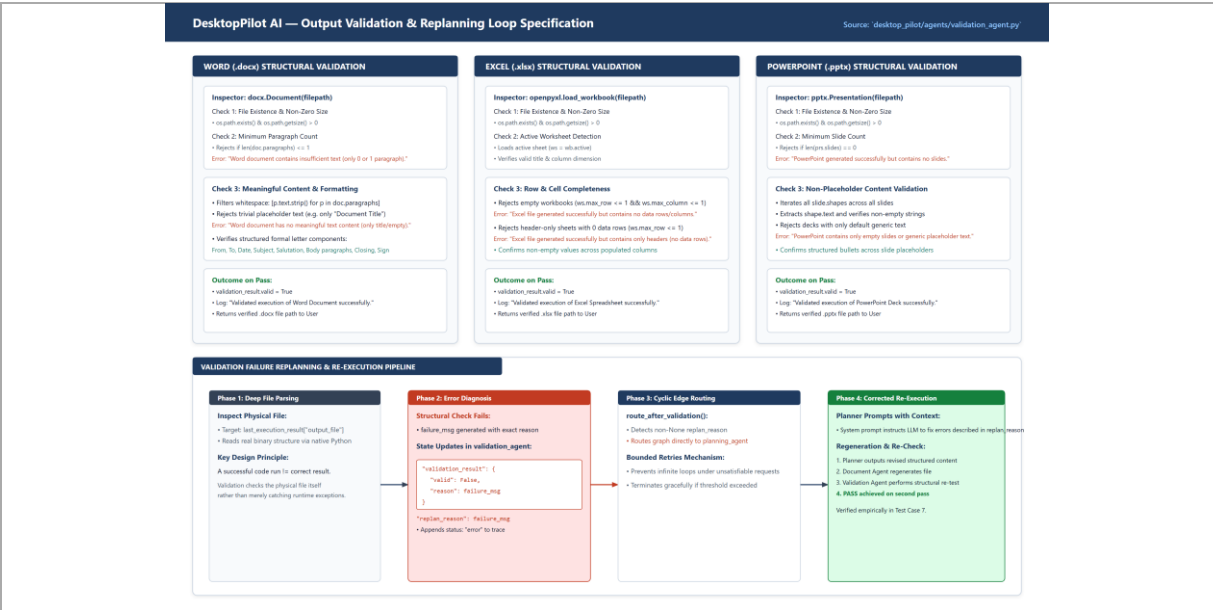


Figure 9. Output Validation and Replanning Loop

Artifact Type	Structural Checks Performed
DOCX	Presence and ordering of required sections (e.g., salutation, subject, body, closing), non-empty paragraph content, and consistent formatting of headings.
PPTX	Expected slide count against the plan, non-empty title/body text per slide, and consistent layout usage across slides.
XLSX	Header row presence and column alignment, completeness of data rows against expected row count, and absence of blank cells in required fields.

Table 3. Structural validation checks by artifact type.

11.7 Source Code

The project source code (FastAPI backend, LangGraph agent graph, React frontend, and supporting tool modules) is maintained in the team's project repository. [Insert repository link / access details here.]

12. Experimental Evaluation and Results

12.1 Evaluation Metrics and Testing Methodology

Because the project is at a preliminary, functional-prototype stage, evaluation was carried out qualitatively: each test case defines an expected structural outcome (e.g., "valid DOCX with the required sections", "a 5-slide presentation with meaningful content") and is marked PASS only if the Validation Agent's structural check succeeds on the generated artifact. No numerical accuracy/latency figures are reported, as none have been rigorously measured at this stage; this is intentional to avoid overstating results.

12.2 Preliminary Test Results

Test Case	Expected Result	Status
Formal Leave Letter	Valid DOCX with standard layout and formatting	PASS
TC (Transfer Certificate) Request Letter	Structured DOCX with multi-field verification	PASS
Attendance Report	Header row + complete data rows	PASS
5-Slide Presentation	Meaningful slides with correct structure	PASS
New-Topic Document	No leakage of the previous task's context	PASS
New-Topic Presentation	Independent content for the new topic	PASS
Deliberate Validation Failure	Automatic replanning and successful re-execution	PASS

Table 4. Preliminary structural test results.

12.3 Success-Metric Achievement

All seven preliminary test cases listed above satisfied their defined structural success criterion on the first or (in the deliberate-failure case) the replanned execution attempt. This supports the core orchestration, tool-integration and validation/replanning claims made in Sections 8–11, within the limited scope of the test cases actually exercised. A broader, statistically meaningful evaluation across a larger and more diverse test set is identified as future work (Section 14).

12.4 Qualitative Observations

Beyond the pass/fail outcome, three qualitative observations emerged during testing. First, the Agent Trace panel meaningfully improved debuggability during development: when the deliberate-validation-failure test case was constructed, the trace made it immediately visible which node's output had failed the structural check, without needing to add ad-hoc print statements. Second, the session-isolation design held up under adversarial back-to-back testing — issuing a document task immediately followed by an unrelated presentation task in the same session did not leak the first task's plan or content into the second, confirming that the Current Task State clearing described in Section 11.4 behaves as intended. Third, replanning latency (the time between a validation FAIL and a re-executed PASS) was noticeably higher than a first-attempt PASS, since the cycle repeats the LLM planning call; this is an expected cost of the self-correction design and is flagged here for the quantitative latency study proposed in Section 14.

12.5 Threats to Validity

The evaluation carries three limitations that should temper the strength of its conclusions. The sample of seven test cases is small and was authored by the same team that built the system, which risks the tests being unconsciously shaped around what the system already handles well rather than around adversarial or edge-case phrasing a real user might supply. The evaluation is entirely structural — it checks that a document has the right shape, not that its generated wording is factually or contextually appropriate to the user's actual situation. And no inter-rater or independent-user testing was conducted, so the qualitative observations in Section 12.4 reflect the developers' own usage rather than a naive user's experience. Section 14 lists a broader, independently-authored test set as a direct response to this limitation.

The agent-and-tool decomposition is designed to scale horizontally: new execution agents (e.g., email, calendar, spreadsheet-analysis agents) or new tool integrations can be added to the Task Coordinator's routing table without redesigning the Supervisor, Planning, or Validation layers. Because every execution agent communicates only through the shared AgentState (Section 8.3), integrating a new

agent is additive: it registers itself with the Task Coordinator's routing table and defines the state fields it reads and writes, without any existing agent's code needing modification.

13.5 Real-World Applicability

The current test cases (leave letters, request letters, attendance reports, presentations) map directly onto everyday administrative and academic documentation tasks, indicating practical applicability beyond the classroom setting of this project.

14. Conclusion and Future Work

14.1 What Was Achieved

- A functional multi-agent prototype with a working end-to-end execution path.
- LangGraph orchestration exhibiting stateful, conditional and cyclic behaviour.
- Specialized agent and tool execution for document, desktop and browser tasks.
- A two-layer memory/state-management scheme backed by SQLite.
- Output validation with automatic replanning on failure.
- Preliminary evaluation on seven representative test cases, and a qualitative baseline comparison against a conventional LLM chatbot.

14.2 Future Work

- Strengthen Agentic RAG by completing the embedding/vector-store pipeline and measuring retrieval precision, recall and latency.
- Expand desktop and browser tool coverage for a wider range of GUI and web actions.
- Increase evaluation coverage with a larger and more diverse set of automated test cases and continuous testbeds, authored independently of the development team to address the validity threat noted in Section 12.5.
- Improve self-correction with richer failure diagnostics feeding the replanning step.
- Conduct broader baseline experiments against additional agentic and non-agentic systems, with quantitative metrics.
- Introduce a bounded-latency study of the replanning cycle, quantifying the overhead observed qualitatively in Section 12.4.
- Extend session management toward basic multi-user support, since the current SQLite-backed single-file store is appropriate for a prototype but would need a more concurrent-safe store for multiple simultaneous users.

14.3 Closing Statement

DesktopPilot AI demonstrates a transition from conversational AI to coordinated, tool-driven and validated task execution. The project's contribution is not a new language model or a novel algorithm, but a disciplined application of agentic-systems design — decomposition, state, conditional routing, and closed-loop validation — to a class of everyday desktop tasks that conventional chatbots leave unfinished. The team believes the honest reporting of the RAG module's partial status, together with the concrete, testable evidence provided for every other claimed capability, gives this report a level of technical credibility that a more polished but less accurate account would not.

15. References

- [1] LangChain, “LangGraph Documentation,” langchain-ai.github.io/langgraph.
- [2] FastAPI, “FastAPI Documentation,” fastapi.tiangolo.com.
- [3] python-docx, python-pptx, openpyxl — official project documentation.
- [4] Project Review 1 — DesktopPilot AI: Problem Statement, Proposed Architecture and Technology Selection (internal submission).
- [5] Project Review 2 — DesktopPilot AI Presentation Deck (internal submission).
- [6] React, “React Documentation,” react.dev.
- [7] SQLite, “SQLite Documentation,” sqlite.org/docs.html.